OBJECT ORIENTED PROGRAMMING

Assignment 02

3 Questions • Total Marks: 100 • Spring 2026

Course Instructor : Ma'am Hina Iqbal

📧: hina.iqbal@lhr.nu.edu.pk

Teacher Assistant (TA) : Syed Saad Ali

📧: l242549@lhr.nu.edu.pk

Section : BSSE-2A/2B

Q1	Q2	Q3
Rational Number System	Custom Text Handling	Calendar Management
35 Marks	35 Marks	30 Marks

Coding Standards & Assignment Guidelines

Naming Conventions

- Use lowerCamelCase for all variables and functions — e.g., `userDetails`, `calculateTotal`
- Use ALL_CAPS for constants — e.g., `PI`, `MAX_SIZE`

Documentation & Comments

- Add comments wherever the logic is non-trivial or needs explanation
- Every function definition must include a documentation comment block — in Visual Studio, place your cursor above the function and press **Ctrl + /** to auto-generate it

Exception Handling

- Handle invalid user input — assume the user can and will enter anything
- Check for null pointers before dereferencing manually managed memory
- Free or delete every allocation that is no longer in use — no memory leaks
- Your program must remain stable under all conditions — no unexpected crashes or undefined behavior

Academic Integrity

- Plagiarism of any kind will result in a **DC referral** — no exceptions
- You may use tools like ChatGPT to *understand* a concept, never to copy a solution

Time Management

- Do not start two days before the deadline or on weekends — this assignment is long and logic heavy
- Read and understand the problem fully first
- Identify patterns, design your logic on paper, then implement
- If anything about a question is unclear, ask in the **WhatsApp group** beforehand — no excuses will be accepted during evaluation

Regarding use of built-in functions and Pointer-based allocation

The use of any built-in functions is strictly prohibited. Arrays must be accessed using pointer-based allocation *() ; using square brackets [] will result in mark deductions

Question 01 Rational Number System

Operator Overloading • GCD Simplification • Rule of Three

Marks: 35

Problem Overview

In mathematics, a rational number is expressed as **numerator / denominator**, for example: `3/4`, `-5/2`, `7/1`. You must design and implement a fully functional **Rational Number System** in C++ that behaves like a built-in numeric type, with automatic simplification and complete operator support.

📌 Non-Negotiable Design Rules

- Denominator must NEVER be zero — throw an exception if attempted.
- Always store in fully reduced (GCD-simplified) form after every operation.
- If denominator is negative, transfer the sign to the numerator.
- No global variables. Proper const-correctness throughout.

1.1 Class Design

```
C++
```

```
class Rational {
private:
    int numerator;     // Can be negative (sign always lives here)
    int denominator;   // Always positive, never zero

public:
    // Constructors, member functions, all operators below ...
};
```

1.2 Constructors

Constructor	Signature / Behavior	Example
`Default`	Creates 0/1. No arguments needed.	Rational r; → 0/1
`Parameterized`	Rational(int num, int den). Validate den≠0, reduce, fix sign.	Rational(4,8) → 1/2 Rational(3,-6) → -1/2
`Single-param`	Rational(int num). Creates num/1.	Rational(5) → 5/1

`Copy Constructor`	Deep copy of another Rational object.	Rational b = a; (independent)

1.3 Member Functions

Signature	Description	Example
`void reduce()`	Simplify using GCD. Must be called internally after every operation.	8/12 → 2/3
`double toDecimal() const`	Return double value of the fraction.	1/4 → 0.25
`Rational reciprocal() const`	Return new Rational with num/den swapped. Throw if numerator is 0.	3/4 → 4/3
`void display() const`	Print in format: numerator/denominator	Prints: 5/6

c++

```
// GCD helper (use this inside reduce()):
int gcd(int a, int b) {
    a = (a < 0) ? -a : a;
    b = (b < 0) ? -b : b;
    return b == 0 ? a : gcd(b, a % b);
}
```

1.4 Operator Overloading

▸ A) Arithmetic Operators + − * /

Each operator must support three forms: `Rational op Rational`, `Rational op int`, and `int op Rational`. All return a NEW reduced Rational object.

📥 Input	📤 Expected Output
`Rational a(1,2), b(1,3);` `cout << a + b;`	`5/6`
`cout << a - b;`	`1/6`

`Rational a(2,3), b(3,5);` `cout << a * b;`	`2/5`
`Rational a(4,5), b(2,3);` `cout << a / b;`	`6/5`
`Rational a(3,4);` `cout << a + 2;`	`11/4`
`cout << 3 + a;`	`15/4`

▸ B) Compound Assignment Operators `+=` `-=` `*=` `/=`

Modify the calling object in-place. Return reference to `*this`. Auto-simplify after each.

📥 Input	📤 Expected Output

`Rational r(1,2);`
`r += Rational(1,3);`
`cout << r;`
	`5/6`

`Rational r(3,4);`
`r -= Rational(1,4);`
`cout << r;`
	`1/2`

`Rational r(2,3);`
`r *= Rational(3,4);`
`cout << r;`
	`1/2`

▸ C) Comparison Operators `==` `!=` `<` `>` `<=` `>=`

Cross-multiply to compare accurately (avoids floating-point errors). Return `bool`.

Formula: `a/b < c/d` is equivalent to `a*d < b*c` (when b,d > 0)

📥 Input	📤 Expected Output
`Rational a(1,2), b(2,4);` `cout << (a == b);`	`1  (true — same value)`
`Rational a(3,4), b(2,3);` `cout << (a > b);`	`1  (true — 0.75 > 0.666...)`
`Rational a(1,3), b(1,2);` `cout << (a != b);`	`1  (true)`

▸ D) Increment & Decrement Operators ++ --

Implement both **prefix** and **postfix** versions. Adds or subtracts `1` (i.e., adds `denominator/denominator` to the fraction).

> 💡 **Prefix vs Postfix**
> Prefix (++r): increment FIRST, then return the updated object (return reference).
> Postfix (r++): save a copy of current value, increment, then return the SAVED copy.

📥 Input	📤 Expected Output

`Rational r(3,4);`
`++r;`
`cout << r;`
	`7/4`

`Rational r(3,4);`
`Rational s = r++;`
`cout << s;`
`cout << r;`
	`3/4  (s has old value)`
	`7/4  (r is incremented)`

`Rational r(7,4);`
`--r;`
`cout << r;`
	`3/4`

▸ E) Function Call Operator ()

When `()` is called on a Rational object, it must return the **reciprocal**.

📥 Input	📤 Expected Output
`Rational r(2,3);` `cout << r();`	`3/2`
`Rational r(5,1);` `cout << r();`	`1/5`
`Rational r(0,1);` `r();`	`Throws: cannot take reciprocal of zero`

▸ F) Stream Operators << >>

Must be **friend functions**. Input reads two integers (num den) and auto-simplifies. Output prints `num/den`.

📥 Input	📤 Expected Output

`Rational r;`
`cin >> r;  // user enters: 4 6`
`cout << r;`
	`2/3`

`cin >> r;  // user enters: 10 5`
`cout << r;`
	`2/1`

`cin >> r;  // user enters: 3 0`	`Error: denominator cannot be zero`

1.5 Driver Program

Your `main()` must be a **menu-driven loop** that tests every single feature and continues until the user selects Exit.

```
c++
```

```
==========================================
         Rational Number System
==========================================
 1.  Add two rational numbers
 2.  Subtract two rational numbers
 3.  Multiply two rational numbers
 4.  Divide two rational numbers
 5.  Compound assignment operations
 6.  Compare two rational numbers
 7.  Convert to decimal
 8.  Reciprocal (member function)
 9.  Prefix & Postfix Increment
10.  Prefix & Postfix Decrement
11.  Function call operator ()
12.  Operations with integers
13.  Exit
------------------------------------------
Enter your choice: _
```

Sample Runs:

📥 Input	📤 Expected Output

`Enter first rational (num den): 1 2`
`Enter second rational (num den): 1 3`
	`Addition:  5/6`
	`Subtraction: 1/6`
	`Multiplication: 1/6`
	`Division:  3/2`

`Enter rational: 3 4` `Prefix  ++ :`	`7/4`

`Enter first rational: 2 4`
`Enter second rational: 1 2`
	`Equal?  Yes`
	`Less than?  No`
	`Greater than? No`

Question 02 Custom Text Handling Class

Dynamic Memory • Rule of Three • String Algorithms

Marks: 35

Problem Overview

C++ provides `std::string` for string manipulation. In this question, you must build your own **custom string class** from scratch — without using `std::string` anywhere internally. Your class, named `MyString`, must dynamically manage its own memory and implement all string operations manually.

🚫 Hard Constraints

- std::string is BANNED inside MyString. Using it = zero for Q2.
- All memory must be allocated with new[] and freed with delete[].
- You MUST implement the Rule of Three: Destructor + Copy Constructor + Copy Assignment.
- The stored char array must always be null-terminated (\0 at end).
- No shallow copies — every copy must allocate fresh memory.

2.1 Class Design

C++

```
class MyString {
private:
    char* str;   // Dynamically allocated char array (null-terminated)
    int length;   // Number of characters NOT counting '\0'

public:
    // All constructors, destructor, member functions, operators ...
};
```

2.2 Constructors, Copy Assignment & Destructor

Method	Behavior	Key Requirement
`MyString()`	Creates empty string ""	Allocate 1 byte for null char. Set length=0.

`MyString(const char* input)`	Deep copy of C-string	Compute length, allocate length+1 bytes, copy char by char.
`MyString(const MyString& other)`	Copy Constructor	Allocate NEW memory. Copy chars. Do NOT copy pointer value.
`operator=(const MyString& other)`	Copy Assignment	Check self-assignment first. Free old memory, allocate new, copy.
`~MyString()`	Destructor	delete[] str. Never double-delete.

💡 **Why Deep Copy Matters**

A shallow copy copies only the pointer address. Both objects then point to the SAME memory block. When one is destroyed, delete[] is called, leaving the other with a dangling pointer. This causes crashes.

Deep copy = always allocate new memory and copy every character one by one.

c++

```
// Deep copy pattern (use inside copy constructor & assignment):
length = other.length;
str = new char[length + 1];       // allocate fresh memory
for (int i = 0; i <= length; i++) // copy chars + null terminator
   str[i] = other.str[i];
```

2.3 Member Functions

Signature	Description	Example
`int lengthOfString() const`	Returns char count (not counting null terminator).	"Hello" → 5
`int countWords() const`	Count space-separated words. Handle multiple consecutive spaces.	"Hello world C++" → 3
`void toUpperCase()`	Convert all chars to uppercase in-place. Use char arithmetic.	"hello" → "HELLO"
`void toLowerCase()`	Convert all chars to lowercase in-place.	"WORLD" → "world"

`void toSentenceCase()`	First char uppercase, all remaining lowercase.	"hELLO WORLD" → "Hello world"
`MyString reverse() const`	Return NEW MyString with chars in reverse order. Caller unchanged.	"abc" → "cba"
`void display() const`	Print the string to stdout.	—

2.4 Operator Overloading

▸ A) Concatenation Operators + and +=

All overloads must allocate new memory dynamically. `+=` must reallocate and free old memory.

Expression	Behavior
`MyString + MyString`	Return new MyString = concatenation of both.
`MyString + const char*`	Append C-string to right. Return new MyString.
`const char* + MyString`	Prepend C-string to left. Must be a friend function.
`MyString += MyString`	Append in-place. Reallocate. Return *this.
`MyString += const char*`	Append C-string in-place. Reallocate.

📥 Input	📤 Expected Output

`MyString a("Hello"), b("World");`
`MyString c = a + " " + b;`
`c.display();`
	`Hello World`

`MyString t("Hello");`
`t += " C++";`
`t.display();`
	`Hello C++`

`MyString t("World");`
`MyString r = "Hello " + t;`
`r.display();`
	`Hello World`

▸ B) Comparison Operators == != < > <= >=

Compare **lexicographically** (dictionary order). Do NOT use `strcmp` — implement character-by-character comparison manually.

📥 Input	📤 Expected Output
`MyString a("Apple"), b("Banana");` `cout << (a < b);`	`1  (A < B in ASCII)`
`MyString a("Hello"), b("Hello");` `cout << (a == b);`	`1  (identical)`
`MyString a("Zebra"), b("Apple");` `cout << (a > b);`	`1  (Z > A in ASCII)`

▸ C) Indexing Operator []

Return a `char&` so the caller can both read and modify. Throw `std::out_of_range/error` for invalid index.

📥 Input	📤 Expected Output
`MyString s("Hello");` `cout << s[1];`	`e`

`MyString s("Hello");`
`s[0] = 'h';`
`s.display();`
	`hello  (character was modified)`

`MyString s("Hi");` `cout << s[10];`	`Throws: std::out_of_range/error`

▸ D) Function Call Operator ()

Returns a **reversed copy** of the string. Same behavior as `reverse()` member function.

📥 Input	📤 Expected Output

`MyString t("Programming");`
`MyString r = t();`
`r.display();`
	`gnimmargorP`

▸ E) Stream Operators << >>

Must be **friend functions**. >> reads a full line, dynamically resizes buffer. << prints via `display()`.

📥 Input	📤 Expected Output

`MyString t;`
`cin >> t;  // user types: Hello World`
`cout << t;`
	`Hello World`

`MyString t;`
`cin >> t;  // user types: (empty line)`
`cout << t.lengthOfString();`
	`0`

2.5 Driver Program

```
c++
```

```
==========================================
   Custom Text Processing Engine
==========================================
 1.  Enter new string
 2.  Display string
 3.  Show length
 4.  Count words
 5.  Convert to uppercase
 6.  Convert to lowercase
 7.  Convert to sentence case
 8.  Reverse string
 9.  Concatenate another string
10.  Compare with another string
11.  Access character using index []
12.  Test assignment operator =
13.  Test function call operator ()
14.  Exit
------------------------------------------
Enter your choice: _
```

Sample Runs:

📥 Input	📤 Expected Output
`Input: hello world` `Choice 5 (Uppercase):`	`HELLO WORLD`
`Input: hELLO WORLD` `Choice 7 (Sentence Case):`	`Hello world`
`First: Hello  |  Second: C++` `Choice 9 (Concatenate):`	`Hello C++`
`First: Apple  |  Second: Banana` `Choice 10 (Compare):`	`Equal? No` `First < Second? Yes`
`Input: Programming` `Choice 8 (Reverse):`	`gnimmargorP`

Question 03 Calendar Management System

Date Arithmetic • Leap Year Logic • Formatted Output

Marks: 30

Problem Overview

Date handling is critical in real-world applications: banking systems, airline reservations, appointment scheduling, and payroll systems. You must design and implement a **CalendarDate** class in C++ that models a valid Gregorian calendar date with full arithmetic, comparison, formatting, and leap-year support.

3.1 Class Design

```
c++
```

```
class CalendarDate {
private:
   int day;    // Valid range: 1 to daysInMonth(month, year)
   int month;  // Valid range: 1 to 12
   int year;   // Any positive year (Gregorian calendar)

public:
   // constructors, validators, arithmetic, operators ...
};
```

3.2 Date Validation Rules

Leap Year Rule	Year	Leap?
Divisible by 4 AND not by 100	2024	✔ Yes
Divisible by 100 (not 400) → NOT leap	1900	✘ No

Divisible by 400 → always leap	2000	✔ Yes
Not divisible by 4	2023	✘ No

```
c++
```

```
// Leap year implementation:
bool isLeapYear() const {
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}
```

3.3 Constructors

Constructor	Behavior	Example
`CalendarDate()`	Default: 01-01-2000	CalendarDate d;
`CalendarDate(int d, int m, int y)`	Validate before storing. Throw or reset to default on invalid.	CalendarDate(29,2,2024) → valid CalendarDate(29,2,2023) → invalid
`CalendarDate(int totalDays)`	Base = 01-01-2000. Add totalDays to base date.	CalendarDate(1) → 02-01-2000 CalendarDate(365) → 31-12-2000
`CalendarDate(const CalendarDate&)`	Copy day, month, year.	Standard deep copy

3.4 Member Functions

Signature	Description	Example
`bool isLeapYear() const`	Returns true if year is a leap year.	2024 → true, 2023 → false
`int daysInMonth(int m, int y) const`	Returns days in month m of year y. Must respect leap year for Feb.	Feb 2024 → 29, Feb 2023 → 28

`bool isValid() const`	Returns true if the stored date is a valid calendar date.	31/02/2023 → false
`int toTotalDays() const`	Convert current date to total days since 01-01-2000. Required for arithmetic.	02-01-2000 → 1
`void addDays(int n)`	Add n days. Handle month/year/leap transitions. Large n allowed.	28-02-2024 + 1 → 29-02-2024
`void subtractDays(int n)`	Subtract n days. Handle backward month/year transitions.	01-01-2025 - 1 → 31-12-2024
`void print(int fmt) const`	fmt=1: DD-MM-YYYY, fmt=2: Month D YYYY, fmt=3: YYYY/MM/DD	fmt=2: March 5, 2026

💡 **toTotalDays() Tip**

Convert date to total days since base, perform arithmetic as integer, then convert back.
This greatly simplifies addDays, subtractDays, and the subtraction operator (Date - Date).
Example: (10-03-2026).toTotalDays() - (05-03-2026).toTotalDays() = 5

3.5 Operator Overloading

▸ A) Arithmetic Operators

Expression	Behavior	Example
`date + int`	Return NEW date after adding n days.	05-03-2026 + 5 → 10-03-2026
`int + date`	Same as above (friend function).	5 + 05-03-2026 → 10-03-2026
`date - int`	Return NEW date after subtracting n days.	10-03-2026 - 5 → 05-03-2026
`date - date`	Return int = chronological difference in days.	10-03-2026 - 05-03-2026 → 5

▸ B) Compound Assignment `+=` `-=`

Modify existing date in place. `d += 5` adds 5 days to `d`. Return `*this`.

▸ C) Increment & Decrement `++` `--`

Both prefix and postfix for `++` and `--`. Move date forward or backward by one day. Handle all boundary cases.

📥 Input	📤 Expected Output

`CalendarDate d(28,2,2024);`
`++d;`
`cout << d;`
	`29-02-2024  (leap year handled)`

`CalendarDate d(31,12,2025);`
`++d;`
`cout << d;`
	`01-01-2026  (year rollover)`

`CalendarDate d(1,3,2024);`
`--d;`
`cout << d;`
	`29-02-2024  (backward into leap Feb)`

`CalendarDate d(5,3,2026);`
`CalendarDate e = d++;`
`cout << e;`
`cout << d;`
	`05-03-2026  (e = old value)`
	`06-03-2026  (d incremented)`

▸ D) Comparison Operators `==` `!=` `<` `>` `<=` `>=`

Compare chronologically. Use `toTotalDays()` on both objects and compare the integers.

📥 Input	📤 Expected Output

`CalendarDate a(1,1,2025),`
`b(1,1,2026);`
`cout << (a < b);`
	`1  (true)`

`CalendarDate a(5,3,2026),`
`b(5,3,2026);`
`cout << (a == b);`
	`1  (true)`

`cout << (a != b);`	`0  (false, same date)`

▸ E) Function Call Operator `()`

When `d()` is called, print the date in **all three supported formats** automatically.

📥 Input	📤 Expected Output

`CalendarDate d(5,3,2026);`
`d();`
	`05-03-2026`
	`March 5, 2026`
	`2026/03/05`

▸ F) Stream Operators << >>

`>>` reads format `DD MM YYYY` and validates before storing. `<<` prints `DD-MM-YYYY` format. Both must be friend functions.

📥 Input	📤 Expected Output

`CalendarDate d;`
`cin >> d;  // user enters: 15 08 2026`
`cout << d;`
	`15-08-2026`

`cin >> d;  // user enters: 31 02 2023`	`Invalid Date! Reset to: 01-01-2000`

`cin >> d;  // user enters: 29 02 2024`
`cout << d;`
	`29-02-2024  (2024 is a leap year)`

3.6 Driver Program

```
c++
```

```
==========================================
   Calendar Management System
==========================================
 1.  Enter a new date
 2.  Display date
 3.  Add days
 4.  Subtract days
 5.  Compare two dates
 6.  Find difference between two dates
 7.  Test increment operators (++ prefix & postfix)
 8.  Test decrement operators (-- prefix & postfix)
 9.  Test compound operators += and -=
10.  Check leap year
11.  Display in all formats (function call operator)
```

```
12.  Exit
-----------------------------------------
Enter your choice: _
```

Sample Runs:

📥 Input	📤 Expected Output
`Enter date (DD MM YYYY): 28 02 2024` `Choice 3 (Add Days): 1`	`New Date: 29-02-2024  ✔ leap handled`
`Enter date: 01 01 2025` `Choice 4 (Subtract Days): 1`	`New Date: 31-12-2024  ✔ year rollback`
`First: 10 03 2026  |  Second: 05 03 2026` `Choice 6 (Difference):`	`Difference: 5 days`
`Enter date: 31 02 2023`	`Invalid Date! Reset to: 01-01-2000`

📋 Submission Format

- Zip your complete folder and name the zip file with your roll number.
- Folder name: 25L-XXXX_A1
- Question files: 25L-XXXX_Q1, 25L-XXXX_Q2, etc.
- Include all necessary files required to run the code (including any .txt files).

- Late submissions: -5 marks per day unless prior approval granted.

Good Luck! (ig)